Backlog del Sprint 4 — Scaps

Panel de administración

> **Sprint 4 · 10 al 23 de agosto de 2026.** Con el catálogo, la ficha, la sesión y el carrito funcionando, este sprint cierra el MVP con el panel de administración: cargar productos con su modelo 3D y sus fotos, editarlos, darlos de baja o reactivarlos, gestionar su galería y elegir el destacado de la landing. La API de administración de productos existe desde el Sprint 1; le faltan las dos subidas de archivos a R2 y borrar el archivo cuando se borra una imagen. En el frontend, el acceso desde el navbar, el listado con sus acciones, el alta, la edición y la galería, más tres piezas que comparten esas pantallas: el diálogo de confirmación, los campos del producto y la subida de archivos. No hay migraciones. Al terminar tiene que estar desplegado: el admin entra al panel, carga una gorra nueva con su `.glb` y sus fotos, y cualquiera la ve en el catálogo y en 3D. El panel es también la herramienta para reponer, antes de la demo, el stock que descuenta la compra simulada.
>
> Cada tarea indica su área entre corchetes, sus labels, qué hacer, el criterio para darla por terminada y, cuando aplica, de qué depende. La forma de trabajo (columnas del tablero, flujo y reglas) está en `Sprint-0-Scaps.pdf` y no se repite acá.

Antes de arrancar: el panel es simple, pero completo. Cumple todo lo que piden el plan y el contrato, con la misma terminación que el resto de la app, y nada más. Queda afuera:

- **Todo lo que depende de las órdenes:** métricas, pedidos y clientes. La hoja del panel en la guía visual muestra un menú lateral con Órdenes y Clientes, indicadores de ventas y las últimas órdenes: nada de eso va, porque las órdenes son de la segunda etapa. `GET /admin/metrics` y `GET /admin/orders` no se usan, y con una sola sección el panel no lleva menú lateral.
- **Gestionar productos desde la tienda:** todo se hace en el panel. El catálogo, la ficha y la landing no cambian para el admin, que los ve como cualquier usuario logueado; lo único que se le suma es el enlace Panel del navbar.
- **La gestión de usuarios:** los admins se siguen creando con el seed.
- **La búsqueda y el filtro por estado del listado:** con cuatro productos no suman nada.
- **La vista previa en 3D dentro del formulario, reordenar imágenes arrastrando y las acciones en lote.**
- **Como deuda técnica,** comprimir las imágenes en el backend y borrar los archivos que quedan huérfanos en R2, como los que sube el admin antes de cancelar un alta o el `.glb` anterior cuando lo reemplaza. La tarea de la subida en la API las anota en `docs/deuda-tecnica.md`.

El bucket de R2, una carpeta por entorno. Todo lo que sube la API va a la carpeta de su entorno dentro de `scaps-assets`: `prod/` en Render y `dev/` en desarrollo. La define la variable `STORAGE_KEY_PREFIX`, que es obligatoria: sin ella la API no arranca, así nadie sube sin querer a otra carpeta. Las fotos y los modelos del seed se quedan en la raíz, como están, y los usan producción y las bases de desarrollo de todo el equipo. Por eso **la API solo borra archivos de su propia carpeta**: borrar desde el panel una imagen del seed quita el registro y deja el archivo. El token de R2 no se puede limitar a una carpeta, así que esta regla es lo que evita que una prueba en

desarrollo borre una foto de producción. Lo que se junte en `dev/` se puede vaciar de una vez desde el panel de Cloudflare. El CORS del bucket no se toca: las subidas pasan por el backend.

Las pruebas se hacen con el admin del seed contra tu base de desarrollo, nunca contra producción: el panel cambia lo que ve cualquier visitante. Eso deja afuera los previews de Vercel, que usan la API y la base de producción (ver el README): el panel se prueba en local. Los archivos que subas probando quedan en `dev/`, y `npm run seed -w @scaps/api` devuelve los productos de demo a como estaban, con su galería.

Y reglas para todas las tareas:

- **Una sola librería nueva:** el cliente de S3, `@aws-sdk/client-s3`, en el backend, para hablar con R2. En el frontend, ninguna: ni plantillas de admin ni librerías de tablas, formularios o modales. Sale con React, Tailwind, React Router y Zustand.
- **Reutilizar, no rehacer.** Se reutilizan y, donde una tarea lo pide, se completan, sin rehacerlos:
  - **En la API:** la administración de productos del Sprint 1, que ya cumple las reglas del contrato (una sola portada por producto, nunca menos de una imagen ni más de 10, un solo destacado y solo entre los activos), y `@Roles('admin')`, que protege una ruta con los guards de sesión y de admin.
  - **Para hablar con la API:** el cliente HTTP (`lib/api.ts`), que ya manda un `FormData` sin forzar el `Content-Type`, y los hooks de `hooks/useApi.ts`: `useApiQuery` para leer, sin `keep`, que es solo para datos públicos, y `useApiAction` para las mutaciones.
  - **Formularios y avisos:** `TextField`, `SubmitButton`, `FormAlert`, `LineNotice` y `focusField`, los del login, el registro y el carrito.
  - **Lo demás:** `formatPrice`, `<ProductImage>`, el navbar, `useSessionStore`, `RequireAdmin` y los tipos `ProductDetail`, `GalleryImage` y `Paginated`.
- **El stock se ve en el panel.** La regla de no mostrar el número es para los clientes: el admin lo ve y lo edita.
- **Plata:** el precio se escribe como en Argentina (`15.999` o `15999,50`) y viaja como string decimal (`"15999.00"`). El frontend nunca hace cuentas con plata: el campo de precio convierte texto a texto, y para mostrar está `formatPrice`.
- **Subidas:** si una subida falla, no se guarda nada y se avisa qué archivo falló.

To Do — se pueden tomar ya (no dependen de nada)

[infra] Tokens de R2 para la API

infra prioridad-alta

- **Tipo:** configuración en Cloudflare y en Render (no es un PR). La toma quien tenga acceso a la cuenta de Cloudflare del bucket.
- **Qué hacer:** crear los dos tokens con los que la API va a escribir en el bucket y cargar las variables que los usan. En el panel de Cloudflare, en R2, se crea un token de la API de R2:
  - **Con el permiso *Object Read & Write* y solo sobre `scaps-assets`** (*Apply to specific buckets only*). Ni permisos de administración ni acceso a otros buckets: la API solo sube y borra archivos.
  - **Dos tokens:** `scaps-api-prod`, para Render, y `scaps-api-dev`, para el equipo. Si el de desarrollo se filtra, se revoca sin tocar producción.
  - **El secreto se muestra una sola vez,** al crear el token, junto con la dirección del servicio para clientes de S3. Si se pierde, no se recupera: se crea otro token y se revoca el anterior.

- **Las variables** ya están, sin valores, en el bloque de almacenamiento de `apps/api/.env.example`; la sexta, `STORAGE_KEY_PREFIX`, la suma la tarea de la subida en la API. Son las mismas en los dos entornos:
  - `STORAGE_ENDPOINT`: la dirección para clientes de S3 que muestra Cloudflare al crear el token, `https://<account-id>.r2.cloudflarestorage.com`.
  - `STORAGE_ACCESS_KEY` y `STORAGE_SECRET_KEY`: las del token de cada entorno.
  - `STORAGE_BUCKET`: `scaps-assets`.
  - `STORAGE_PUBLIC_URL`: `https://pub-6128219e47234ffb824474fa3df9636d.r2.dev`, sin barra al final.
  - `STORAGE_KEY_PREFIX`: la carpeta del entorno, `prod/` en Render y `dev/` en desarrollo (ver "Antes de arrancar").
- **Dónde van:** las de producción, en el panel de variables de entorno de Render. Las de desarrollo, en el `apps/api/.env` de cada integrante, que git ya ignora; se pasan por un mensaje privado, nunca por el chat del grupo, un issue ni el repositorio.
- **Hecho cuando:** existen los dos tokens, cada uno con *Object Read & Write* solo sobre `scaps-assets`; Render tiene las seis variables con el token de producción y `STORAGE_KEY_PREFIX` en `prod/`; cada integrante tiene en su `.env` las de desarrollo, con `STORAGE_KEY_PREFIX` en `dev/`; y ningún secreto aparece en el repositorio ni en un chat compartido.

> **Por qué va primero:** sin estas credenciales la subida de archivos en la API no se puede probar, y de ella dependen el alta, la edición y la galería. Es configuración de un rato: conviene hacerla el primer día. Mientras tanto, las otras tres tareas de To Do no dependen de nada y se pueden tomar en paralelo.

[dashboard] Enlace al panel y rutas nuevas

dashboard prioridad-alta

- **Qué hacer:** dejar el panel a un clic para los admins y declarar las dos pantallas nuevas, así las tareas del alta y la edición encuentran su ruta hecha y no tocan el router.
  - **El enlace Panel en el navbar:** un `NavLink` a `/admin` dentro de `NavLinks` (`components/Navbar.tsx`), después de Catálogo y con el mismo `navLinkClassName`. `NavLinks` arma la barra en escritorio y el menú desplegable en el celular, así que con sumarlo una vez aparece en los dos, y el menú se cierra al elegirlo como con los otros enlaces. Solo para admins: va si hay sesión y `user.es_admin` de `useSessionStore`, que es lo que lee `RequireAdmin`.
  - **Se marca como sección activa** en `/admin` y en las dos pantallas nuevas. `NavLink` ya lo hace con las rutas que cuelgan de la suya, siempre que no lleve la prop `end`.
  - **Las dos rutas nuevas,** `/admin/productos/nuevo` y `/admin/productos/:id`, en `App.tsx`, cada una envuelta en `RequireAdmin` como `/admin`, con su página vacía en `pages/` (por ejemplo, `NuevoProducto.tsx` y `EditarProducto.tsx`, en castellano como las demás): el título ("Nuevo producto", "Editar producto") y un enlace **Volver al panel**. El contenido lo ponen las tareas del alta y la edición.
- **Hecho cuando:** con el admin del seed, el enlace se ve, lleva a `/admin` y queda marcado en las tres rutas del panel; con un usuario común y sin sesión, no aparece; entrar a `/admin/productos/nuevo` sin sesión manda al login y, al ingresar con el admin, vuelve ahí; con un usuario común, las dos rutas nuevas se comportan igual que `/admin` hoy; y en los cuatro anchos de prueba el navbar se ve completo, sin scroll horizontal, con Panel dentro del menú en el celular.

[dashboard] Diálogo de confirmación

dashboard prioridad-alta

- **Qué hacer:** un componente `<ConfirmDialog>` (`components/ConfirmDialog.tsx`) para pedir confirmación antes de una acción que no se deshace sola. Lo usan dos tareas: dar de baja un producto, en el listado, y borrar una imagen, en la galería. Va en una tarea propia para que las dos lo encuentren hecho y ninguna arme el suyo.
  - **Con un `<dialog>` del navegador, como `PhotoLightbox`:** se abre con `showModal()` desde un `useEffect` y se entera del cierre con `onClose`, sin librerías. El navegador atrapa el foco adentro, cierra con Esc y devuelve el foco al botón que lo abrió, y el CSS global ya frena el scroll de la página mientras hay un `<dialog>` abierto.
  - **Recibe** el título, el texto, la etiqueta del botón de confirmar (por ejemplo, “Dar de baja”) y la acción. Cancelar cierra sin hacer nada.
  - **El foco arranca en Cancelar,** así un Enter apurado no confirma: va primero en el DOM, que es donde el navegador pone el foco al abrir, como Cerrar en `PhotoLightbox`.
  - **La acción puede tardar:** mientras corre, los dos botones quedan con `aria-disabled` y Esc no cierra (el evento `cancel` se cancela con `preventDefault()`). Si sale bien, el diálogo se cierra; si falla, queda abierto con el error adentro, en un `FormAlert`, para reintentar o cancelar.
  - **Estética:** una card como la del login, el botón de confirmar con el estilo de acción primaria y Cancelar como botón secundario. Sin rojo: el único rojo de la app marca un campo inválido, y el peligro lo dice el texto del botón. En el celular ocupa el ancho con un margen, con los botones uno debajo del otro.
- **Hecho cuando:** montado de prueba en cualquier página (y sacado antes de abrir el Pull Request), con una acción que tarda dos segundos y otra que falla: se abre con el foco en Cancelar; con Tab no se llega a nada de la página de atrás, que tampoco scrollea; Esc y Cancelar cierran sin ejecutar nada; mientras la acción corre, los botones no responden y Esc no cierra; si falla, se ve el error y el diálogo sigue abierto; al cerrarse, el foco vuelve al botón que lo abrió; un lector de pantalla anuncia el título (el `<dialog>` lleva `aria-labelledby`); y en los cuatro anchos de prueba se ve completo.

> **Por qué no `window.confirm`:** no se puede estilar, así que rompe la estética oscura, no deja mostrar el error de una acción que falló y cada navegador lo dibuja a su manera.

[dashboard] Campos del producto y precio en formato argentino

dashboard prioridad-alta

- **Qué hacer:** un componente con los cuatro campos de datos de un producto —nombre, descripción, precio y stock— y sus validaciones (por ejemplo, `components/ProductFields.tsx`). Lo usan el alta y la edición, así que va en una tarea propia, como el selector de cantidad del Sprint 3: las dos pantallas lo encuentran hecho y validan igual. No llama a la API: recibe valores iniciales opcionales (los usa la edición) y le entrega a la pantalla los valores ya en el formato del contrato, si son válidos y cuál es el primer campo con error. Los archivos no van acá: el `.glb` y las imágenes los resuelve cada pantalla con la subida de archivos.
- **Los campos:** nombre, precio y stock con `<TextField>`, que ya trae la etiqueta, el error debajo y el `aria-invalid` y el `aria-describedby`; la descripción es un `<textarea>` con las mismas clases, porque `TextField` solo arma un `<input>`.
  - **Nombre:** obligatorio; se recortan los espacios de los costados.
  - **Descripción:** opcional. Vacía es “sin descripción”: el alta no la manda y la edición manda `null`, que es como el contrato la borra.
  - **Precio:** obligatorio, con las reglas de abajo e `inputMode="decimal"`.

  - **Stock:** un entero de 0 a 1.000.000, el tope del contrato, con `inputMode="numeric"`, que en el celular abre el teclado numérico. Los puntos, si los hay, separan miles como en el precio: `1.000` es mil y `1.0` es un error.
- **El precio se escribe como en Argentina y viaja como en el contrato.** Una función lo pasa del campo al string del contrato, y otra hace el camino inverso para precargar la edición: por ejemplo, `parsePriceInput` y `formatPriceInput`, en `lib/format-price.ts`, al lado de `formatPrice`.
  - **El punto separa los miles y la coma, los centavos:** `15999`, `15.999` y `15999,00` dan `"15999.00"`; `15999,5` y `15.999,50` dan `"15999.50"`. El signo `$` y los espacios se descartan.
  - **Los puntos tienen que separar grupos de tres cifras.** `15.99` no es válido, y el error lo explica: “Para los centavos usá coma: 15,99”. Así un precio escrito a la inglesa no se guarda como otro número.
  - **Hasta dos decimales, mayor que cero y hasta `99999999.99`,** como el contrato.
  - **De texto a texto, sin pasar por número.** `parseFloat` y `toFixed` redondean y arrastran los errores del `float`; con cadenas y una expresión regular no hay cuentas, y se respeta la regla de la plata.
  - **Para precargar,** `"15999.00"` se muestra `15.999` y `"15999.50"`, `15.999,50`: lo mismo que `formatPrice`, sin el `$`. Ojo: el espacio que sigue al `$` es uno no separable (U+00A0), no uno común.
- **Cuándo muestra los errores, como el login y el registro:** se calculan siempre, pero se ven recién cuando la pantalla intenta guardar —se lo indica con una prop, como el `submitAttempted` de esas pantallas— y de ahí en más se van solos al corregir el campo. Al intentar guardar con errores, la pantalla le pasa el foco al primer campo inválido con `focusField`.
- **Hecho cuando:** montado de prueba en cualquier página (y sacado antes de abrir el Pull Request), con un botón que simula guardar: los ejemplos de arriba dan exactamente esos resultados, y `0`, `-5`, `15,999`, `1.5` y `100.000.000` dan error con su motivo; precargado con `"15999.50"`, muestra `15.999,50` y, sin tocar nada, entrega otra vez `"15999.50"`; el stock acepta `0` y `1.000.000` y rechaza `1.000.001`, `-1` y `2,5`; un nombre vacío o con solo espacios da error; antes de tocar el botón no se ve ningún error, al tocarlo aparecen los de todos los campos inválidos y el foco va al primero, y al corregir un campo su error se va solo; en el celular, el precio y el stock abren el teclado numérico; se maneja entero con el teclado; y en los cuatro anchos de prueba se ve completo.

Backlog — esperan que se libere una dependencia

[dashboard] Subida de archivos a R2 en la API

dashboard prioridad-alta

- **Depende de:** Tokens de R2 para la API.
- **Qué hacer:** implementar `POST /uploads/model` y `POST /uploads/image` tal cual los define la sección 5 del contrato, en un módulo nuevo, `src/uploads/`, armado como los demás. El controller lleva `@Controller('uploads')` y `@Roles('admin')` en la clase, como `AdminProductsController`, y el módulo importa `AuthModule`, que aporta el `JwtService` de los guards, como hace `ProductsModule`. Suma la única librería nueva del sprint: `npm install @aws-sdk/client-s3 -w @scaps/api`.
- **El cliente de R2,** en un módulo propio, `src/storage/` (`StorageModule` y `StorageService`), armado como `src/prisma/`: exporta el servicio y no es global, así lo importan el módulo de subidas y, en la tarea del borrado, el de productos.
  - **R2 habla el mismo protocolo que S3,** así que se usa el `S3Client` oficial, con `region: 'auto'`, el `endpoint` de `STORAGE_ENDPOINT` y las credenciales de `STORAGE_ACCESS_KEY` y `STORAGE_SECRET_KEY`.
  - **Las seis variables son obligatorias,** como `JWT_SECRET` en `AuthModule`: si falta alguna, la API no arranca y el error dice cuál y remite a `apps/api/.env.example`.

  - **Cada archivo se guarda con un nombre único:** `STORAGE_KEY_PREFIX`, un uuid y la extensión (`dev/3f2a…c9.webp`), con `randomUUID()` de `node:crypto`. El nombre original no se usa: puede repetirse o traer caracteres raros.
  - **Con su tipo** (`ContentType`): `model/gltf-binary` para el `.glb` y el de cada imagen para las fotos. Si no, R2 los sirve como un binario genérico.
  - **Devuelve la URL pública:** `STORAGE_PUBLIC_URL`, una barra y el nombre del archivo.
- **Los dos endpoints, con sus reglas:**
  - **Reciben el archivo** en el campo `file` con el `FileInterceptor` de `@nestjs/platform-express`, que ya está instalado, en memoria: de ahí va directo a R2. El tipo `Express.Multer.File` viene en `@types/multer`, que no está: en vez de sumar otro paquete, una interfaz chica con lo que se usa (`originalname`, `mimetype`, `size` y `buffer`).
  - **Cada uno con su tope,** en la opción `limits.fileSize` del interceptor: 20 MB para el `.glb` y 2 MB para las imágenes.
  - **`POST /uploads/model`** acepta solo `.glb`: por la extensión, sin importar mayúsculas, y por los primeros 4 bytes, que en un `.glb` siempre dicen `glTF`. El tipo que manda el navegador no se mira, porque con un `.glb` suele mandar `application/octet-stream`.
  - **`POST /uploads/image`** acepta solo `image/png`, `image/jpeg` e `image/webp`, y la extensión sale del tipo (`png`, `jpg`, `webp`), no del nombre original.
  - **Responden `201` con `{ url }`,** como el contrato.
- **Un archivo que no corresponde nunca termina en `500`:** sin archivo, `400`; con un tipo que no corresponde, `400` ("El archivo tiene que ser un modelo .glb", "La imagen tiene que ser PNG, JPG o WebP"). Pasado el tope, NestJS responde por su cuenta `413` con un mensaje en inglés ("File too large"): un filtro de excepciones chico en cada endpoint lo cambia por `400` con el tope en castellano ("La imagen supera los 2 MB"), porque el contrato responde `400` a todo request inválido y el frontend muestra el mensaje tal cual. Si el que falla es R2, sí es un error del servidor: `500`, con el detalle en el log y nunca las credenciales (el frontend muestra su propio mensaje para cualquier 5xx).
- **Documentarlo donde ya está lo demás:** en `apps/api/.env.example`, el bloque de almacenamiento deja de decir "todavía sin usar" y suma `STORAGE_KEY_PREFIX`; en el README, "Variables de entorno" pasa a pedir las de almacenamiento y las variables de Render suman las seis; y `docs/deuda-tecnica.md` suma, con el formato de sus entradas, la compresión de imágenes y los archivos huérfanos en R2 (ver "Antes de arrancar").
- **Hecho cuando:** con los casos en `apps/api/requests/uploads.http`, como los demás archivos de esa carpeta (REST Client arma el *multipart* y toma el archivo con `< ./ruta`), con el token del admin del seed y contra tu entorno de desarrollo: uno de los `.glb` de `docs/glb/comprimidos/` responde `201`, y su URL, puesta como `glb_url` de un producto de tu base desde `prisma studio`, se ve en **Ver en 3D**; el mismo `.glb` mandado como `application/octet-stream` también entra; una imagen PNG, una JPG y una WebP responden `201` y sus URLs se abren en el navegador como imagen; un archivo de texto renombrado a `.glb`, un `.gltf`, un PDF como imagen y un pedido sin archivo dan `400`; una imagen de 3 MB y un archivo de más de 20 MB como modelo dan `400` con el tope en el mensaje; sin token responde `401` y con el de un usuario común, `403`; los archivos quedan en la carpeta `dev/` del bucket; sin `STORAGE_KEY_PREFIX` la API no arranca y dice por qué; `npm test -w @scaps/api` sigue en verde; y el `.env.example`, el README y la deuda técnica quedan al día, sin ningún valor real en el repositorio.

> **Por qué el tope va en el interceptor:** el contrato pide aplicarlo en el borde del request, antes de leer el cuerpo. Un validador que mira el archivo después, como el `MaxFileSizeValidator` de NestJS, recibe primero los megas enteros en memoria y recién ahí los rechaza; con `limits.fileSize`, la lectura se corta apenas el archivo se pasa.

[dashboard] Borrar de R2 el archivo de una imagen

dashboard prioridad-alta

- **Depende de:** Subida de archivos a R2 en la API.
- **Qué hacer:** completar `DELETE /products/:id/images/:imageId` con lo que pide el contrato y el Sprint 1 no podía hacer, porque no había cliente de R2: borrar también el archivo. Hoy `ProductImagesService.remove` borra solo el registro, como dice su comentario. La regla de no borrar la única imagen, el reemplazo de la portada y el resto del endpoint quedan como están.
  - **Solo archivos de la carpeta del entorno** (ver "Antes de arrancar"): si la URL de la imagen empieza con `STORAGE_PUBLIC_URL`, una barra y `STORAGE_KEY_PREFIX`, se borra ese archivo; si no —una foto del seed, en la raíz del bucket, o la de otro entorno—, se borra solo el registro.
  - **Primero el registro, después el archivo.** `remove` borra el registro dentro de `prisma.serializable(...)`; el archivo se borra cuando la transacción ya terminó, con la URL que se leyó adentro (hoy esa consulta no la trae). Al revés, si fallara la base después de borrar el archivo, quedaría en la galería una imagen que apunta a la nada. Así, lo peor que puede pasar es un archivo huérfano, que es deuda ya anotada.
  - **Si R2 falla al borrar, el endpoint responde `204` igual** y deja el error en el log: para el admin, la imagen ya no está, que es lo que pidió.
  - **El borrado va en `StorageService`,** como un método más al lado del de subir, y `ProductsModule` importa `StorageModule`: el cliente de R2 queda en un solo lugar.
- **Hecho cuando:** en tu entorno de desarrollo, con una imagen subida con `POST /uploads/image` y agregada a un producto con `POST /products/:id/images`, borrarla responde `204` y el archivo desaparece de `dev/`; borrar una foto del seed responde `204`, la saca de la galería y el archivo sigue en la raíz del bucket; borrar la única imagen de un producto sigue dando `409`, como hoy; con un `STORAGE_ENDPOINT` inventado, el borrado responde `204` igual y el error queda en el log; y los casos de `apps/api/requests/admin-gallery.http` siguen respondiendo lo que anotan.

[dashboard] Subida de archivos desde el frontend

dashboard prioridad-alta

- **Depende de:** Subida de archivos a R2 en la API.
- **Qué hacer:** dejar listo, en un solo lugar, todo lo que necesitan las pantallas del panel para subir un archivo: el alta sube el `.glb` y las imágenes, la edición reemplaza el `.glb` y la galería suma imágenes. Va en una tarea propia para que las tres lo encuentren hecho.
  - **El cliente HTTP no se toca:** `lib/api.ts` ya manda un `FormData` tal cual y sin `Content-Type`, que lo arma el navegador con su `boundary`. Quedó preparado para las subidas, como anota su comentario.
  - **Dos funciones,** `uploadModel(file)` y `uploadImage(file)` (por ejemplo, en `lib/uploads.ts`), que arman el `FormData` con el campo `file`, llaman a `api.post` y devuelven la URL.
  - **Las mismas reglas que el backend, antes de subir:** el modelo, `.glb` y hasta 20 MB; las imágenes, PNG, JPG o WebP y hasta 2 MB. En constantes (`MAX_MODEL_SIZE`, `MAX_IMAGE_SIZE`) y en dos funciones que devuelven el motivo del rechazo ("foto.png supera los 2 MB"), así el aviso llega sin esperar la subida. El backend valida igual: esto es comodidad, no seguridad.

  - **Un campo para elegir archivos,** `<FileField>` (`components/FileField.tsx`), que usan las tres pantallas: el `<input type="file">` con una etiqueta como la de `TextField`, el texto de ayuda, el `accept` que corresponde y, si se pide, selección múltiple. Valida con las reglas de arriba, le pasa a la pantalla solo los archivos válidos y muestra el motivo de los que rechaza, o el error que le pase la pantalla al guardar (por ejemplo, “Elegí un modelo .glb”). Es el input nativo, con el botón estilado con el modificador `file:` de Tailwind: así anda con el teclado sin trucos. Después de cada elección vacía su valor, para que se pueda volver a elegir el mismo archivo.
- **Hecho cuando:** montado de prueba en cualquier página (y sacado antes de abrir el Pull Request), contra tu API local: elegir uno de los `.glb` de `docs/glb/comprimidos/` y subirlo devuelve una URL de `dev/` que se abre; lo mismo con una imagen; una imagen de 3 MB, un `.txt` y un `.gltf` se rechazan con su motivo, y la pestaña *Network* no muestra ningún pedido; un error que le pasa la pantalla se ve en el mismo lugar; en *Network*, la subida sale como `multipart/form-data` con su `boundary`; con el backend apagado aparece un error legible; elegir dos veces seguidas el mismo archivo funciona; y el campo se maneja entero con el teclado y se ve completo en los cuatro anchos de prueba.

> **Ojo con las subidas largas:** un `.glb` de 20 MB tarda con una conexión lenta. El cliente HTTP no corta los pedidos por tiempo, así que la subida termina sola, pero a los 5 segundos aparece el aviso global de servidor dormido (`SlowServerNotice`), que mide cualquier pedido. No se toca: cada pantalla muestra su propio estado.

[dashboard] Listado de productos del panel

dashboard prioridad-alta

- **Depende de:** Enlace al panel y rutas nuevas.
- **Qué hacer:** construir la pantalla `/admin` en `pages/Admin.tsx`, que hoy muestra solo el título “Admin”, con todos los productos, activos e inactivos, de `GET /admin/products`, pedidos con `useApiQuery`. Es la portada del panel: tiene que dejar ver de un vistazo qué está a la venta, qué se quedó sin stock y cuál es el destacado.
  - **Encabezado:** el título “Productos”, con la cantidad total (`meta.total`), y el botón **Nuevo producto**, que lleva a `/admin/productos/nuevo`.
  - **Por cada producto:** la imagen principal (la de `es_principal: true` de su galería) con `<ProductImage aspect="square">` en un área chica; el nombre; el precio con `formatPrice`; el stock, con el número (ver “Antes de arrancar”); si está activo; si es el destacado; y el enlace **Editar**, a `/admin/productos/:id`. Las demás acciones las suma la tarea de baja, reactivación y destacado.
  - **Sin stock, a primera vista:** es lo único accionable del panel (reponer o dar de baja). El stock en 0 lleva el `AlertIcon` delante y el texto Sin stock: así marca la app lo que hay que atender, con el ícono y no con un color, porque es monocroma.
  - **Inactivo y destacado:** un producto dado de baja se ve apagado y con el cartel **Inactivo**, y se puede editar igual; el destacado lleva el cartel **Destacado**. Los dos, con la forma de los carteles de la tienda, como `SoldOutBadge`.
  - **Si no hay destacado** — `GET /products/featured`, también con `useApiQuery`, responde `404` —, un aviso arriba del listado con `LineNotice`: “No hay un producto destacado: la landing muestra la imagen fija hasta que elijas uno”. Lo pide el contrato. Si ese pedido falla por otro motivo, el aviso no aparece y el listado sigue funcionando.
  - **Paginado con Cargar más, como el catálogo:** el `PAGE_SIZE` y el `useLoadMore` del catálogo son locales de `pages/Catalogo.tsx` y no se tocan; el listado lleva su propia tanda de 24 y una versión más simple, porque no tiene filtros. Del catálogo se copia cómo se comporta el botón: `aria-disabled` y “Cargando…” mientras pide, el pie “Mostrando 24 de 30 productos” y el foco al pie cuando llega la última tanda. El orden es el de la API por defecto, los más recientes primero.

  - **Avisos de otras pantallas:** si la navegación llega con un aviso en su `state` (el alta vuelve acá con “Producto creado”), se muestra arriba del listado y se limpia del historial, así no vuelve a aparecer al recargar.
  - **Estados:** cargando; error con **Reintentar**, que llama al `reload()` de `useApiQuery`; y vacío, “Todavía no hay productos”, con **Nuevo producto**. Si a un producto le falta la imagen principal, se usa la primera de la galería: un dato faltante no rompe la fila.
- **Tabla en escritorio, tarjetas en el resto:** desde `lg` (1024 px), una tabla armada como las líneas del carrito (`pages/Carrito.tsx`): una grilla con su fila de encabezados en mayúsculas chicas sobre `scaps-card-highlight` y las filas separadas por `scaps-border-row`. Debajo, cada producto es una tarjeta, con la imagen a la izquierda, los datos al lado y las acciones abajo, a todo el ancho. Nada de una tabla que obligue a scrollear de costado.
- **Hecho cuando:** con el admin del seed contra tu base de desarrollo, se ven los cuatro productos con imagen, nombre, precio, stock y estado, y el destacado marcado; con el stock de uno en 0 desde `prisma studio`, aparece Sin stock y se distingue sin buscarlo; con uno en `activo: false`, se ve apagado e Inactivo; sin ningún `destacado` en `true`, aparece el aviso; con la tanda en 2, Cargar más trae el resto y el botón desaparece (y vuelve a 24 antes del Pull Request); con el backend apagado aparece el error con Reintentar; Editar lleva a `/admin/productos/:id` con el id de ese producto; y en los cuatro anchos de prueba se ve completo, sin scroll horizontal, con la tabla en 1440 y tarjetas en los otros tres.

[dashboard] Baja, reactivación y destacado desde el listado

dashboard prioridad-alta

- **Depende de:** Listado de productos del panel + Diálogo de confirmación.
- **Qué hacer:** sumar a cada producto del listado las acciones que faltan, cada una contra su endpoint del contrato y con su `useApiAction`:
  - **Dar de baja,** en los activos: pide confirmación con `<ConfirmDialog>` y llama a `DELETE /products/:id`. El texto dice qué pasa: “Deja de verse en el catálogo y en su ficha. Se puede reactivar después”. Si es el destacado, lo dice también: “Es el destacado: la landing va a mostrar la imagen fija hasta que elijas otro”.
  - **Reactivar,** en los inactivos: `PATCH /products/:id` con `{ "activo": true }`, sin confirmación, porque no rompe nada. Vuelve al catálogo, pero no como destacado: el contrato no lo vuelve a destacar.
  - **Destacar,** en los activos que no lo son: `PUT /products/featured` con `{ "product_id": … }`. Reemplaza al anterior, así que el cartel Destacado pasa a este producto y se va del otro. En los inactivos no aparece, porque el contrato solo deja destacar productos activos.
- **El listado se actualiza sin recargarse,** porque se perderían las tandas de Cargar más: cada acción cambia la fila con lo que responde la API. El `DELETE` responde `204`, sin cuerpo, así que la baja marca la fila como inactiva y sin destacar. Después de dar de baja el destacado o de destacar otro, se vuelve a consultar `GET /products/featured` para mostrar u ocultar el aviso.
- **Mientras una acción está en curso** se deshabilitan los botones de ese producto, no la pantalla entera. **Si falla** (por ejemplo, un `409` al destacar un producto que alguien dio de baja desde otra pestaña), el mensaje del backend se ve en esa fila con `LineNotice`, como en las líneas del carrito, y se vuelve a pedir ese producto con `GET /admin/products/:id`, así queda al día.
- **En el celular,** las acciones van en la tarjeta, debajo de los datos, a todo el ancho y con 44 px de alto.
- **Hecho cuando:** dar de baja pide confirmación y, al confirmar, el producto queda Inactivo y deja de verse en el catálogo; dar de baja el destacado hace aparecer el aviso, y la landing muestra la imagen fija; reactivar lo devuelve al catálogo sin el cartel Destacado; destacar otro mueve el cartel, saca el

aviso si estaba, y la landing muestra ese producto en 3D; Destacar no aparece en los inactivos; con la tanda del listado en 2 y la segunda cargada, una acción no vuelve a la primera; un doble clic no manda dos pedidos; y en los cuatro anchos de prueba las acciones se ven completas.

[dashboard] Alta de producto

dashboard prioridad-alta

- **Depende de:** Campos del producto y precio en formato argentino + Subida de archivos desde el frontend + Listado de productos del panel.
- **Qué hacer:** construir la pantalla `/admin/productos/nuevo` (la ruta y su página vacía las dejó la tarea del enlace al panel): un solo formulario con los datos, el modelo 3D y las imágenes, que al guardar crea el producto completo con un solo `POST /products`, como dice el contrato.
  - **Los datos,** con el componente de los campos del producto.
  - **El modelo 3D,** con `<FileField>`: un `.glb`, obligatorio, con la ayuda “Comprimido con Draco, hasta 20 MB”. El contrato pide subirlo ya comprimido, y se comprime como en el Sprint 1: `npx @gltf-transform/cli draco entrada.glb salida.glb`.
  - **Las imágenes, de 1 a 10,** elegidas con `<FileField>` en modo múltiple y en una o varias tandas; si se eligen más de las que entran, se suman hasta 10 y se avisa. Cada una va en una fila con su vista previa (`<ProductImage>` con una URL local de `URL.createObjectURL`, que se libera con `URL.revokeObjectURL` al quitarla o al salir), su **texto alternativo**, obligatorio, una opción **Portada** (exactamente una; de entrada, la primera, y si se quita la portada, pasa a serlo la primera que quede) y **Quitar**.
  - **El orden de la galería es el de la lista.** Para cambiarlo después está la galería de la edición.
  - **El texto alternativo describe esa foto,** no repite el nombre del producto (“Gorra negra vista de frente”): lo leen los lectores de pantalla, y por eso el modelo de datos lo pide por imagen.
- **Al tocar Crear producto**, un `SubmitButton`:
  - **Primero se valida todo,** sin llamar a la API: los campos, que haya un `.glb`, que haya entre 1 y 10 imágenes y que cada una tenga su texto alternativo. Los errores se ven junto a cada campo y el foco va al primero, como en el login.
  - **Después se suben los archivos, de a uno:** el `.glb` y las imágenes en orden, con el formulario deshabilitado y el estado a la vista en el botón (“Subiendo el modelo…”, “Subiendo imagen 2 de 4…”).
  - **Por último, el `POST /products`** con los datos, la `glb_url` y las `imagenes`, cada una con su `url`, su `alt`, su `orden` según la lista, desde 0, y `es_principal` en la portada.
  - **Si una subida falla, el producto no se crea:** se frena ahí, sin el `POST`, y un `FormAlert` dice qué archivo falló y por qué. El formulario vuelve a habilitarse con todo lo cargado.
  - **Al reintentar no se sube de nuevo lo que ya subió:** el formulario guarda la URL de cada archivo subido y solo sube los que faltan; si se cambia o se quita un archivo, su URL se descarta. Si el que falla es el `POST`, se reintenta solo el `POST`.
  - **Si sale bien,** vuelve a `/admin` con el aviso “Producto creado”, y el producto nuevo aparece primero, activo y sin destacar.
- **Cancelar** vuelve al panel sin crear nada. Lo que ya se haya subido queda huérfano en R2: es deuda técnica (ver “Antes de arrancar”).
- **En el celular:** todo en una columna, y cada imagen apilada: la vista previa y Quitar arriba, el texto alternativo y Portada abajo.
- **Hecho cuando:** con el admin del seed contra tu base de desarrollo, crear un producto con un `.glb` y tres imágenes lo deja primero en el listado; su ficha muestra la galería en ese orden, la card del catálogo muestra la portada elegida y **Ver en 3D** carga su modelo; los archivos quedan en `dev/`; en

Network se ven cuatro subidas y un solo `POST /products`; sin `.glb`, sin imágenes, con una imagen sin texto alternativo o con un precio inválido, el error se ve en su campo y no sale ningún pedido; bloqueando `/uploads/image` desde las DevTools (*Network request blocking*), el modelo sube, la primera imagen falla, el producto no se crea y el aviso nombra esa imagen; al desbloquear y reintentar, el modelo no se vuelve a subir; y en los cuatro anchos de prueba se ve completo, sin scroll horizontal.

> **Por qué de a uno y no todos a la vez:** así el aviso dice exactamente qué archivo falló y el reintento sabe cuáles faltan. Son once archivos como mucho: la diferencia de tiempo no se nota.

[dashboard] Edición de producto

dashboard prioridad-alta

- **Depende de:** Enlace al panel y rutas nuevas + Campos del producto y precio en formato argentino + Subida de archivos desde el frontend.
- **Qué hacer:** construir la pantalla `/admin/productos/:id` (la ruta y su página vacía las dejó la tarea del enlace al panel), con el formulario precargado con `GET /admin/products/:id`, pedido con `useApiQuery`, que trae también los inactivos.
  - **Arriba, lo que no se edita acá:** el nombre como título, si está activo y si es el destacado (eso se cambia desde el listado), y el **slug**, fijo y a la vista, con una línea que aclare que no cambia aunque cambie el nombre, así la URL de la ficha queda estable. El contrato no deja editarlo.
  - **Los datos,** con el componente de los campos del producto, precargados: el precio `"15999.00"` aparece como `15.999`.
  - **El modelo 3D:** el nombre del archivo actual (el final de su `glb_url`) y un `<FileField>` para **Reemplazar modelo**, con la ayuda “Comprimido con Draco, hasta 20 MB”.
- **Guardar cambios manda solo lo que cambió:** un `PATCH /products/:id` con los campos cuyo valor, ya en el formato del contrato, es distinto del cargado. Si hay un modelo nuevo, primero se sube y su URL va como `glb_url` en el mismo `PATCH`; si la subida falla, no se guarda nada y se avisa. La descripción borrada va como `null`.
- **Sin cambios, el botón queda deshabilitado.** Es un `SubmitButton`, que hoy solo se deshabilita mientras carga: se le suma una prop opcional para este caso, con `aria-disabled` como el resto de la app, sin cambiar cómo se ve en el login y el registro.
- **Al guardar,** el formulario se deshabilita hasta que responde la API. Si falla, un `FormAlert` muestra el mensaje del backend y lo escrito no se pierde. Si sale bien, la pantalla toma lo que devuelve el `PATCH` como los nuevos valores cargados y muestra “Cambios guardados” junto al botón; el `.glb` anterior, si se reemplazó, queda huérfano en R2 (ver “Antes de arrancar”).
- **Estados:** cargando; “Producto no encontrado”, con **Volver al panel**, si la API responde `404` o `400` (un id que no existe o que no es un uuid, escrito a mano en la URL); y error con **Reintentar**.
- **Las imágenes no van en este formulario:** se gestionan en la galería, que tiene su propia tarea y se monta debajo, en esta misma pantalla.
- **En el celular:** todo en una columna, con **Guardar cambios** a todo el ancho.
- **Hecho cuando:** con el admin del seed contra tu base de desarrollo, `/admin/productos/:id` abre con los datos de ese producto y su slug fijo; cambiar solo el stock manda un `PATCH` con solo `stock` (se ve en *Network*); cambiar el nombre no cambia el slug ni la URL de la ficha; borrar la descripción la saca de la ficha; reemplazar el `.glb` hace que **Ver en 3D** muestre el modelo nuevo; sin cambios, el botón está deshabilitado; un precio inválido no sale; un producto inactivo también se edita; `/admin/productos/no-existe` muestra “Producto no encontrado”; el login y el registro siguen viéndose igual; y en los cuatro anchos de prueba se ve completo.

[dashboard] Galería de imágenes en la edición

dashboard prioridad-alta

- **Depende de:** Edición de producto + Diálogo de confirmación.
- **Qué hacer:** sumar a la pantalla de edición, debajo del formulario, la sección **Imágenes**, que gestiona la galería del producto con los endpoints del contrato. Cada acción va directo a su endpoint y se guarda en el momento, sin el Guardar cambios del formulario: la sección lo aclara en una línea.
  - **Las imágenes,** en el orden de `orden`, como ya las manda la API, cada una con su miniatura (`<ProductImage>`), su texto alternativo y sus acciones.
  - **Agregar:** con `<FileField>`, de a una, más su texto alternativo, obligatorio, y el botón **Agregar**, que sube la imagen y la suma con `POST /products/:id/images` mandando solo `url` y `alt`: sin `orden`, el backend la pone al final, y sin `es_principal`, no la hace portada. Con 10 imágenes, Agregar se deshabilita: es el tope del contrato, y el backend responde `409` a la undécima.
  - **Editar el texto alternativo** en la misma fila, con su botón **Guardar**, que se habilita cuando el texto cambió: `PATCH /products/:id/images/:imageId` con `{ alt }`. Vacío no se puede guardar.
  - **Elegir la portada:** la portada lleva el cartel **Portada**, y las demás, el botón **Hacer portada**: `PATCH` con `{ "es_principal": true }`. El backend desmarca la anterior.
  - **Cambiar el orden sin arrastrar:** botones **Subir** y **Bajar** en cada imagen, con un `aria-label` que diga qué hacen. Al mover una, se arma el orden nuevo y se manda un `PATCH` con `{ orden }` a cada imagen cuya posición cambió, numeradas desde 0. Así se corrigen solos los huecos que dejan los borrados.
  - **Borrar,** con `<ConfirmDialog>` (“La imagen sale de la galería y no se puede recuperar”) y `DELETE /products/:id/images/:imageId`. Si es la portada, el diálogo avisa que la portada pasa a ser la primera que queda: el backend la elige solo. La única imagen no se puede borrar —el backend responde `409`—, así que ahí Borrar está deshabilitado, con el motivo a la vista (“El producto necesita al menos una imagen”).
- **Después de cada acción, la galería se vuelve a pedir** con `GET /admin/products/:id`: la portada y el orden los resuelve el backend, y así la pantalla muestra siempre lo que quedó guardado. Se actualiza solo la galería: lo que se esté editando en el formulario de arriba no se pisa.
- **Mientras una acción está en curso** se deshabilitan los controles de esa imagen; al mover, los de toda la galería, porque cambian varias a la vez. Si algo falla, el mensaje del backend se ve en la sección y la galería se vuelve a pedir.
- **En el celular:** cada imagen es una tarjeta, con la miniatura y el texto alternativo arriba y las acciones abajo, a todo el ancho.
- **Hecho cuando:** con el admin del seed contra tu base de desarrollo, agregar una imagen la suma al final de la galería y de la ficha; editar un texto alternativo lo guarda; elegir otra portada cambia la imagen de la card en el catálogo; subir y bajar una imagen cambia el orden en la ficha; borrar una imagen pide confirmación y la saca de la ficha; borrar la portada deja como portada a la primera que queda; con una sola imagen, Borrar está deshabilitado y se ve el motivo; con 10 imágenes no se puede agregar otra; recargar la página muestra exactamente lo mismo que antes de recargar; y en los cuatro anchos de prueba se ve completa, sin scroll horizontal.